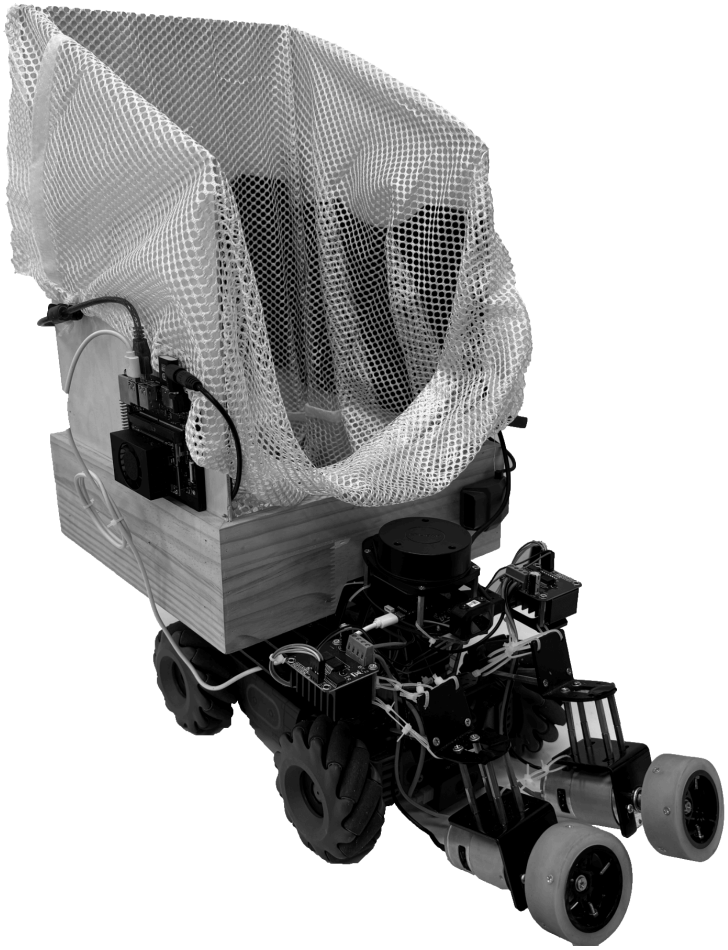
CourtSweeper Mstr

User Guide v1.2

2026.05



Contents

1. Introduction

CourtSweeper is a vision-guided autonomous ball retrieval robot designed for collecting scattered tennis balls on tennis courts. Equipped with an NVIDIA Jetson Orin NX edge computing unit running real-time YOLO object detection, a dual-roller intake mechanism for efficient ball collection, and an iOS companion app for remote monitoring and intervention, CourtSweeper delivers end-to-end autonomous court maintenance.

This document is the **CourtSweeper User Guide**, intended for two audiences:

- **Regular Users**: After receiving the product, follow this manual to assemble the hardware, download the iOS app, and begin operating — no programming experience required.
- **Developers**: Users with experience in Python, ROS2, and deep learning may follow this manual to set up a complete development environment for secondary development and feature extension.

The document is organised as follows:

1. Section 2 lists all parts and materials included in the shipping box
2. Section 3 provides a step-by-step hardware assembly tutorial with photographs
3. Section 4 is for regular users: app download, connection, and operation guide
4. Section 5 is for developers: Jetson flashing, ROS2 environment, SDK configuration, training pipeline
5. Section 6 covers system startup and operational workflows
6. Section 7 lists common issues and diagnostic procedures
7. The Appendix contains quick-reference tables (pin mapping, UDP commands, Wi-Fi modes)

2. Package Contents

CourtSweeper ships as a kit of disassembled parts. Upon unboxing, please verify that all items listed below are present:

Table 1: Bill of Materials

Part No.	Component	Qty
A01	DJI RoboMaster EP chassis (incl. Mecanum wheels ×4)	1
A02	DJI EP Intelligent Battery (3S LiPo, 10.8V, 2400mAh)	1
A03	RoboMaster EP HD camera (integrated)	1

Part No.	Component	Qty
B01	NVIDIA Jetson Orin NX (incl. power adapter)	1
B02	RPLidar A1M8 (incl. USB-serial cable)	1
B03	Arduino UNO + USB-B data cable	1
C01	775 D-shaft DC motor	2
C02	BTS7960 high-current motor driver module	2
C03	3S LiPo battery (11.1V, 5200mAh, 40C)	1
C04	Low-voltage buzzer BX100	1
D01	60mm rubber friction wheel (D-bore)	2
D02	30mm extended brass coupling	2
D03	Dual-roller mounting bracket (incl. screws)	1
E01	Dupont wire set (male-to-female, various)	1
E02	XT60 power cable / terminal block	1

3. Hardware Assembly

+ **WARNING:** Ensure all devices are powered off before assembly. Work on a clean, flat surface to avoid losing small parts.

3.1. Chassis Setup

The RoboMaster EP chassis arrives with Mecanum wheels pre-installed. Only the following steps are required:

1. Insert the EP Intelligent Battery into the rear battery bay; a click indicates proper seating.
2. Press and hold the chassis power button for 2 seconds to turn it on. Verify the battery indicator LEDs illuminate.

Figure 2: DJI RoboMaster EP chassis

3.2. Dual-Roller Intake Assembly

The intake mechanism is the core mechanical subsystem of CourtSweeper, consisting of two 775 motors, two 60mm rubber wheels, two couplings, and a mounting bracket.

Step 2.1 — Coupling installation. Slide the 30mm brass coupling onto the D-shaft of each 775 motor and tighten the set screw with a hex key. Ensure the coupling is fully seated against the shaft shoulder with no axial play.

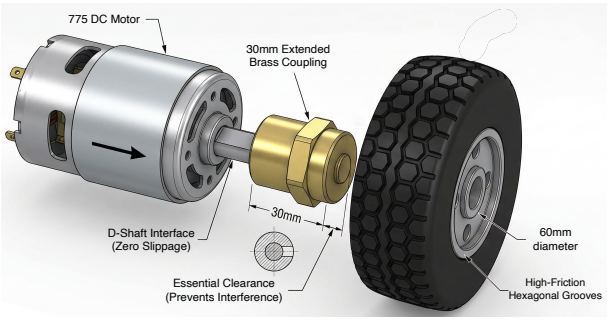


Figure 3: Coupling-to-motor installation detail

Step 2.2 — Wheel installation. Press the 60mm rubber friction wheel onto the opposite D-bore of each coupling and tighten the set screw to secure.

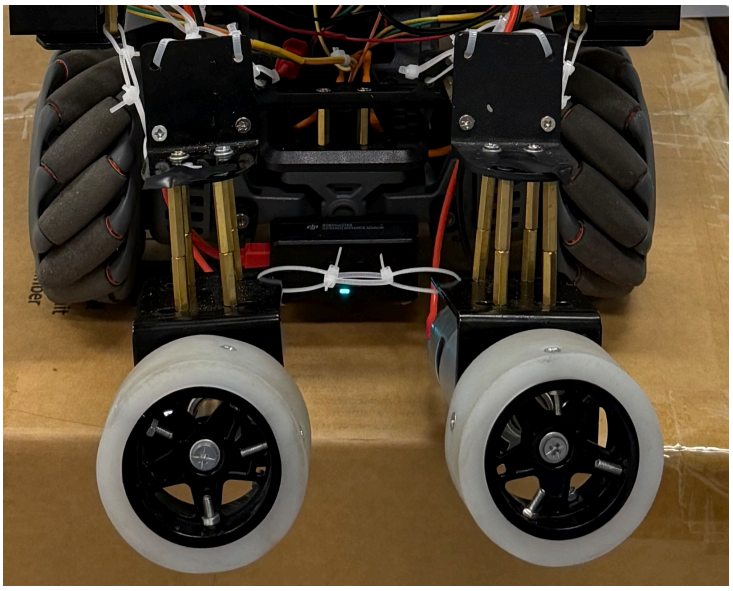


Figure 4: Front view of the dual-roller intake mechanism

Step 2.3 — Bracket attachment. Mount both motor-wheel assemblies onto the dual-roller bracket so the two rubber wheels face each other, with a gap approximately equal to a tennis ball diameter. Fasten the motors using M3 screws through the bracket holes.

Step 2.4 — Mounting to chassis. Attach the entire intake bracket to the front underside of the EP chassis using M4 screws through the pre-drilled mounting holes.

3.3. Motor Driver Wiring

The BTS7960 driver modules require connections to both the Arduino UNO logic pins and the 3S LiPo battery power. Pin assignments are listed in Table 2.

Table 2: Arduino pin assignments for dual BTS7960 drivers

Motor	EN (Enable)	RPWM (Forward)	LPWM (Reverse)
Left Motor	D4	D5	D6
Right Motor	D7	D9	D10

Wiring steps:

1. **Logic side:** Use Dupont wires to connect each BTS7960's EN, RPWM, and LPWM to the corresponding Arduino pins (see Table 2). Both modules share the Arduino 5V and GND.
2. **Power side:** Connect the 3S LiPo battery positive and negative terminals to the power input of both BTS7960 modules in parallel via XT60 connectors. Connect each BTS7960 output to the corresponding 775 motor.

3. **Ground bonding:** Connect Arduino GND to BTS7960 GND to establish a common reference for logic signals.

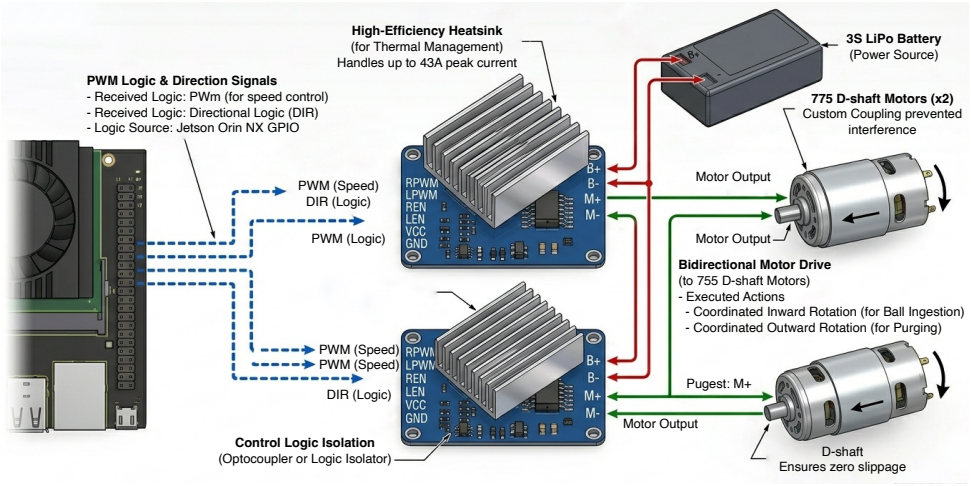


Figure 5: BTS7960 motor driver wiring diagram

+ **WARNING:** Always insert the low-voltage buzzer (BX100) into the 3S LiPo balance lead during wiring to monitor cell voltage and prevent over-discharge damage.

Figure 6: EP chassis battery and 3S LiPo motor battery

3.4. LiDAR & Jetson Orin NX Installation

Secure the RPLidar A1M8 to the rear upper mounting bracket of the chassis using screws. Connect the USB-serial adapter cable to an available USB port on the Jetson Orin NX.

Mount the Jetson Orin NX module on the mid-deck of the chassis using screws or cable ties. Connect the 19V power adapter to the Jetson DC power jack.

RPLidar A1M8 sensor

NVIDIA Jetson Orin NX edge computing unit

3.5. Power Distribution Topology

CourtSweeper employs an isolated dual-battery power scheme:

1. **EP chassis battery:** Supplies the chassis drive motors and camera independently.
2. **3S LiPo battery:** Dedicated to the 775 intake motors and BTS7960 drivers.
3. **Jetson 19V adapter:** Provides independent power for the computing unit.

Common ground requirement: The GND planes of the Arduino, BTS7960 drivers, and Jetson must be connected to ensure reliable PWM logic signal transmission.

3.6. Pre-Flight Checklist

Before first power-up, verify each item:

1. Mecanum wheels are securely fastened, no looseness
2. Coupling set screws are tightened
3. Rubber wheels rotate freely without rubbing the bracket
4. All terminal connections are secure, no exposed conductors
5. 3S LiPo voltage ≥ 11.1V (BX100 not alarming)
6. EP battery fully charged (≥ 3 indicator bars)
7. LiDAR window is clean and unit rotates freely
8. Jetson cooling fan is operational
9. Arduino USB cable is connected to Jetson

4. Quick Start (Regular Users)

This chapter is for users who want to operate the robot out of the box. Following the steps below, you can complete your first drive within 10 minutes.

4.1. Download the iOS App

Open the App Store on your iPhone or iPad, search for **"CourtSweeper"**, and install the official app.

4.2. Connect to the Robot

1. Confirm the robot is powered on (Jetson green LED solid, EP chassis battery indicator lit).
2. Open the Wi-Fi settings on your iOS device, select the Jetson's broadcasted hotspot (SSID format: `CourtSweeper-XXXX`), enter the factory default password `courtsweeper`.
3. Launch the CourtSweeper app. It should auto-detect and connect to the robot. Once connected, the main interface displays the live video stream.

4.3. App Interface Overview

Main HUD: The central area renders the real-time MJPEG video stream from the robot's camera. The feed is overlaid with YOLO-detected tennis ball bounding boxes and a crosshair reticle, giving the operator real-time perception feedback.

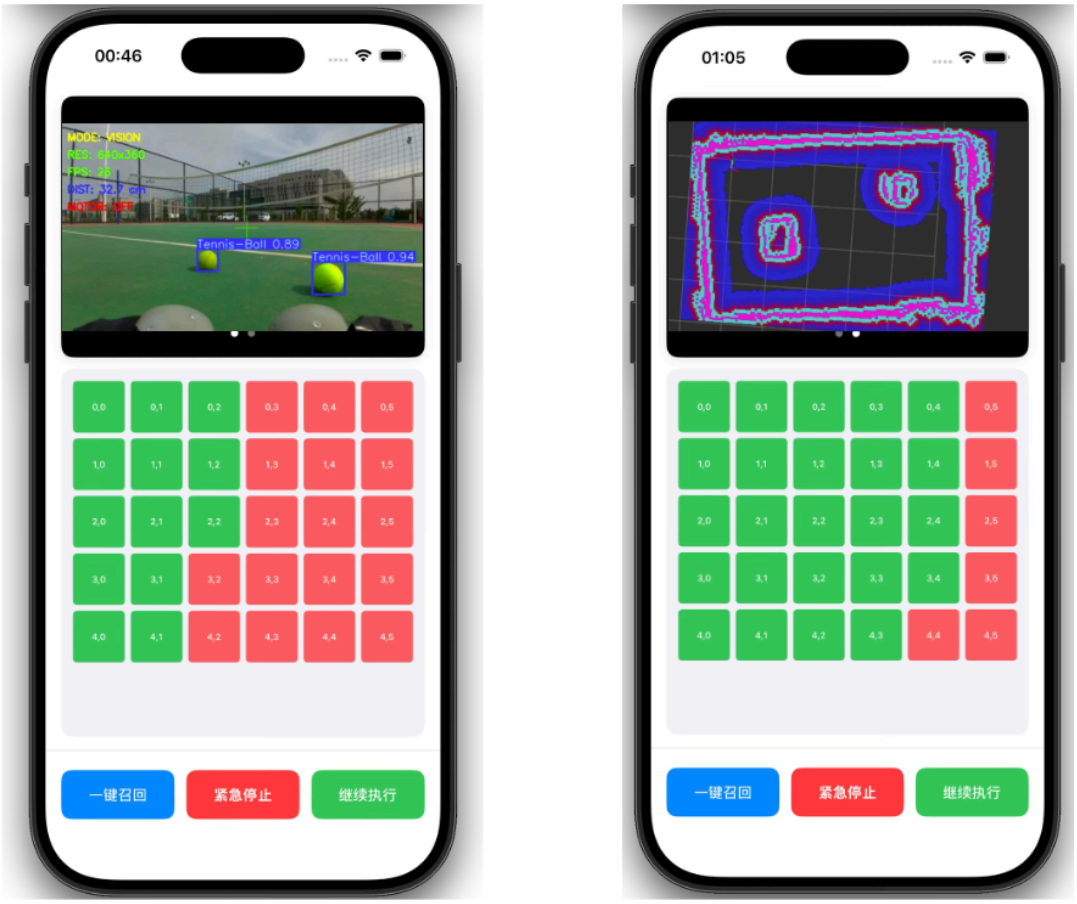


Figure 7: The left shows a live camera feed with YOLO-detected tennis ball bounding boxes and crosshair reticle; while the right is a top-down court map view showing the robot's position, coverage path, and detected ball locations during autonomous mission execution

Virtual Joysticks (bottom-left):

1. Left joystick controls chassis translational motion (forward / backward / lateral strafe).
2. Right joystick controls chassis rotation (clockwise / counterclockwise).

Control Buttons (bottom-right):

1. **AUTO**: Switch to fully autonomous mode — the robot plans and executes ball retrieval autonomously.
2. **MANUAL**: Switch to manual teleoperation mode.
3. **RECALL**: One-touch recall — the robot navigates back to the origin pose via Nav2.
4. **E-STOP**: Emergency stop — immediately brakes and suspends all autonomous logic.

Telemetry Panel (top): Displays battery voltage, Jetson CPU/GPU temperature, and the robot's current FSM state (IDLE / SEARCHING / TRACKING / INGESTING).

4.4. First Operation

1. Ensure the surface is flat and free of obstacles.
2. Place the robot at the starting position of the court.
3. Tap **AUTO** in the app and observe the robot begin its autonomous coverage scan (Boustrophedon S-shaped sweep path).
4. To take manual control at any time, tap **MANUAL** and use the virtual joysticks.
5. When the robot is full or you wish to stop, tap **RECALL** to have the robot return to the origin.

During autonomous operation, the app provides a top-down court map showing the robot's current position, the Boustrophedon coverage path, and detected ball locations.

5. Developer Environment Setup

This chapter is for developers who wish to extend CourtSweeper. You will configure a complete Jetson development environment including the RoboMaster-SDK-Ultra communication stack, ROS2 navigation framework, and YOLO training pipeline.

5.1. Prerequisites

Before you begin, ensure you have:

1. A macOS or Ubuntu host machine for SSH access to the Jetson and YOLO model training
2. NVIDIA Jetson Orin NX (preloaded with Ubuntu 22.04 + JetPack 6.0)
3. Python 3.10+
4. Basic Linux command-line proficiency

5.2. Jetson Flashing (JetPack 6.0)

If your Jetson does not have a preloaded OS or needs a factory reset, follow these steps:

1. Install NVIDIA SDK Manager on the host machine (Ubuntu 22.04 x86_64):

```
sudo apt install nvidia-sdk-manager
sdkmanager
```

2. Connect the Jetson to the host machine via USB-C cable and put it into Recovery Mode (hold the REC button while powering on).
3. In SDK Manager, select `Jetson Orin NX`, and check `JetPack 6.0` along with `Jetson Linux`, `CUDA Toolkit`, `cuDNN`, and `TensorRT`.
4. Wait for the flashing to complete (approximately 30 minutes). Complete the Ubuntu initial setup on first boot.

5.3. ROS2 Humble Installation

Execute the following commands on the Jetson to install ROS2 Humble:

```
sudo apt update && sudo apt install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
sudo apt install -y software-properties-common
sudo add-apt-repository -y universe
sudo apt update && sudo apt install -y curl
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/
ros.key -o /usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/
keyrings/ros-archive-keyring.gpg] http://packages.ros.org/ros2/ubuntu
$(. /etc/os-release && echo $UBUNTU_CODENAME) main" | sudo tee /etc/
apt/sources.list.d/ros2.list > /dev/null
sudo apt update
sudo apt install -y ros-humble-desktop ros-dev-tools
```

Add the environment setup to `~/.bashrc`:

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

Verify the installation:

```
ros2 run demo_nodes_cpp talker
```

5.4. Clone the CourtSweeper Repository

```
git clone https://github.com/RamessesN/CourtSweeper.git ~/CourtSweeper
cd ~/CourtSweeper
```

5.5. Python Dependency Installation

Install the project's Python dependencies on the Jetson:

```
# Core dependencies
pip install --break-system-packages numpy opencv-python pyserial
simple-pid

# YOLO inference
pip install --break-system-packages ultralytics

# LiDAR driver
pip install --break-system-packages rplidar-roboticia

# RoboMaster-SDK-Ultra (core chassis communication library)
pip install --break-system-packages robomaster-sdk-ultra

# ROS2 utilities
pip install --break-system-packages transforms3d
```

`robomaster-sdk-ultra` (https://github.com/RamessesN/Robomaster-SDK-Ultra) is a community-maintained fork of the official DJI RoboMaster SDK, updated for compatibility with Python 3.10+ on modern JetPack releases. The official SDK has not received updates since 2021 and does not support Python ≥ 3.9.

5.6. RoboMaster-SDK-Ultra Configuration

`robomaster-sdk-ultra` (https://github.com/RamessesN/Robomaster-SDK-Ultra) is CourtSweeper's high-performance Python SDK encapsulating all EP chassis communication. The Wi-Fi connection mode must be specified during initialisation:

1. **AP mode:** The Jetson connects to the robot as a hotspot (used for iOS app direct connection and dataset collection).
2. **STA mode:** Both robot and Jetson connect to the same Wi-Fi router (used for SLAM mapping and autonomous navigation).

Example code (`chassis_ctrl.py`):

```
from robomaster import robot

ep_robot = robot.Robot()
# AP mode: Jetson acts as hotspot
ep_robot.initialize(conn_type="ap")
```

```
# STA mode: robot joins router network
# ep_robot.initialize(conn_type="sta", sn="robot_sn")

ep_robot.set_robot_mode(mode="free")
ep_chassis = ep_robot.chassis
```

Wi-Fi mode summary:

Table 3: Wi-Fi operation modes

Mode	SDK Parameter	Use Case
AP	`conn_type="ap"`	iOS direct connection, dataset collection, manual teleop
STA	`conn_type="sta"`	SLAM mapping, Nav2 autonomous navigation

5.7. Arduino Firmware Flashing

The dual-roller intake motors are driven by the Arduino UNO via PWM. The firmware source (`src/roller/roller_setting.ino`) is located in the CourtSweeper repository.

1. Connect the Arduino UNO to the Jetson (or your laptop) via USB-B cable.
2. Install the Arduino IDE or CLI:

```
sudo apt install -y arduino
```

3. Open `src/roller/roller_setting.ino`, select Tool → Port → `/dev/ttyUSB0` (or `/dev/ttyACM0`), and choose the `Arduino Uno` board.
4. Click "Upload" to flash the firmware.

Firmware capabilities (for full firmware design and driver code, see HSDD §4.2):

1. Receives CSV-formatted commands `"L,R\n"` via serial (115200 bps, 8N1)
2. Parses left/right wheel speeds (range −255 to 255) and outputs corresponding PWM
3. Built-in 500ms watchdog: halts all motors on communication loss

5.8. YOLO Model Training Pipeline

CourtSweeper uses YOLOv6-Nano for tennis ball detection. For the complete training hyperparameters, loss functions, and model architecture, see HSDD §4.1 and the Theoretical Manual. The vision module source is at `src/vision/`. If you need to retrain the model with your own dataset, follow these steps:

Step 1 — Dataset collection. Switch the robot to AP mode and run the collection tool:

```
python src/vision/training/dataset_collect.py
```

Use the camera preview to aim: press `s` to capture a photo. Collect at least 500 images under varying lighting and distances. Images are automatically saved to `./dataset/tennis_v2/`.

Step 2 — Dataset annotation. Annotate the captured images with bounding boxes using LabelImg or Roboflow. Export in YOLO format (one `.txt` file per image).

Step 3 — Train the model.

```
python src/vision/training/model_training.py
```

After training, the model file is saved at `src/vision/mlmodel/yolo26n.pt`.

Step 4 — Model conversion (PyTorch → ONNX → TensorRT). For edge deployment on Jetson, convert the model to an FP16 TensorRT engine:

```
# PyTorch → ONNX
python src/vision/mlmodel/pt2onnx.py

# ONNX → TensorRT (FP16)
trtexec --onnx=yolo26n.onnx \
        --saveEngine=yolo26n.engine \
        --fp16
```

5.9. Verification

Run the following commands in separate terminals to confirm each subsystem is operational:

```
# Terminal 1: Launch ROS2 bridge (chassis + video stream)
python src/chassis/chassis_ctrl.py

# Terminal 2: Launch LiDAR driver
python src/lidar/lidar_parse.py

# Terminal 3: Launch IR distance sensor subscriber
python src/distance/distance_sub.py
```

```
# Terminal 4: Launch video inference node
python src/vision/video_node.py
```

If all terminals run without errors and ROS2 topics `/odom`, `/scan`, and `/camera/image/compressed` produce output, the development environment is correctly configured.

6. Running the System

The complete CourtSweeper operational workflow consists of three phases: power-on → mapping → autonomous navigation.

6.1. Power-On Sequence

Table 4: Power-on sequence

Step	Action	Confirmation Signal
1	Insert 3S LiPo (BX100 buzzer)	BX100 displays voltage ≥ 11.1V
2	Insert EP battery	Chassis battery indicator ≥ 3 bars
3	Connect Jetson 19V power	Fan starts, green LED flashing
4	Connect Arduino USB to Jetson	`ls /dev/ttyUSB0` visible
5	Wait for Jetson boot	Wi-Fi hotspot appears / SSH login succeeds

6.2. SLAM Mapping

For first use (or after changing venues), an environment map must be built. The mapping workflow is driven by `map_generation.py` (https://github.com/RamessesN/CourtSweeper/tree/main/ROS/map). For the SLAM and Nav2 architecture design, see HSDD §3.

1. Configure the robot Wi-Fi to STA mode (both robot and Jetson connect to the same router).
2. Start mapping:

```
ros2 launch slam_toolbox online_async_launch.py
python src/../resource/map/map_generation.py
```

1. Use the iOS app in **MANUAL** mode to drive the robot across the entire court. `slam_toolbox` will build a 2D occupancy grid map in real time.
2. Once the area is fully covered, press Ctrl+C to stop mapping. The map is automatically saved as a ROS2 map file pair (`.pgm` + `.yaml`).

Table 5: SLAM mapping workflow

Step	Command	Description
1	`ros2 launch slam_toolbox online_async_launch.py`	Start SLAM node
2	`python map_generation.py`	Start mapping control node
3	Manual teleop across full court	Using App MANUAL mode
4	Ctrl+C stop mapping	Map saved as `map.pgm` / `map.yaml`

6.3. Autonomous Navigation

After mapping is complete, start the Nav2 autonomous navigation stack:

```
ros2 launch nav2_bringup navigation_launch.py map:=./map.yaml
python src/../resource/map/map_navigation.py
```

1. The robot's high-level FSM (Figure 8) automatically transitions to the SEARCHING state and follows the Boustrophedon S-shaped coverage path across the court.
2. When YOLO detects a tennis ball, the FSM transitions to TRACKING. The dual-PID controller aligns the chassis with the target and approaches.
3. Within 200 mm range, the FSM enters INGESTING: roller motors pre-spin, and the chassis drives forward to collect the ball.
4. Upon full coverage or receipt of a RECALL command, Nav2 plans the shortest path back to the origin.

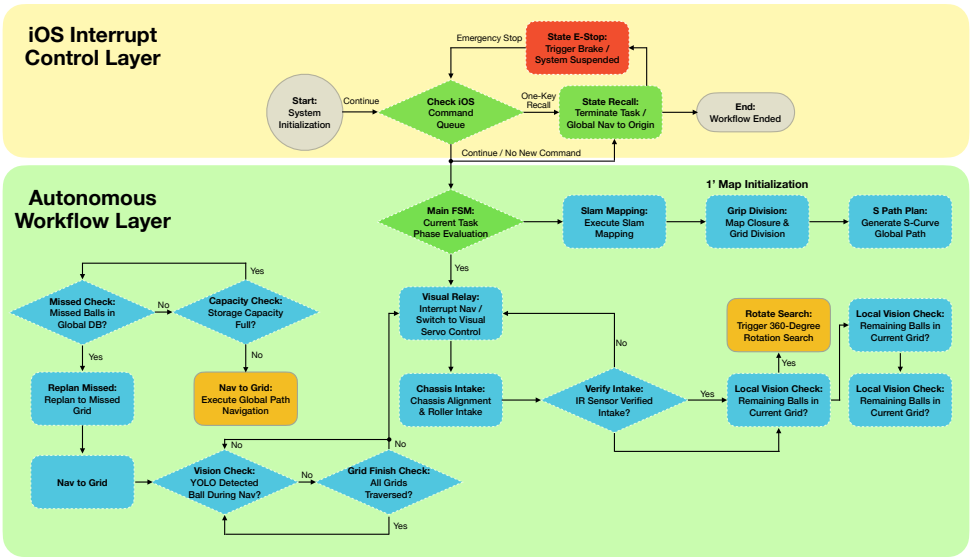


Figure 8: CourtSweeper full operational workflow: SLAM mapping → Nav2 path planning → dual-PID tracking → roller ingestion

6.4. Common Launch Commands Quick Reference

Table 6: Common launch commands quick reference

Scenario	Command	Notes
SLAM Mapping	`ros2 launch slam_toolbox online_async_launch.py && python map_generation.py`	Requires STA mode
Autonomous Navigation	`ros2 launch nav2_bringup navigation_launch.py map:=./map.yaml && python map_navigation.py`	Requires STA mode
Manual Teleop	`python src/chassis/chassis_ctrl.py`	Requires AP mode + iOS app
Test Video Only	`python src/vision/video_node.py`	ROS2 not required
Test LiDAR Only	`python src/lidar/lidar_parse.py`	Requires ROS2
Test Roller Motors	`python src/roller/roller_drive.py`	Requires Arduino connected

7. Troubleshooting

The following table lists common issues encountered during CourtSweeper operation and their diagnostic procedures.

Table 7: Common troubleshooting table

Issue	Possible Cause	Solution
Motors won't spin	Arduino not connected / firmware not flashed / BTS7960 wiring incorrect	Check USB serial visibility: `ls /dev/ttyUSB*`; re-flash Arduino firmware; verify wiring against Table 2
LiDAR produces no data	USB cable loose / serial port permissions	Execute `sudo chmod 666 /dev/ttyUSB0`; verify device detection: `lsusb | grep CP210x`
Video lag / black screen	Wi-Fi bandwidth insufficient / frame rate too high	Move closer to Wi-Fi hotspot; lower resolution to 720P; close other bandwidth-consuming devices
Serial communication dropouts	Loose Dupont wires / serial port contention	Inspect physical connections; check with `lsof /dev/ttyUSB0`; re-plug USB cable

Issue	Possible Cause	Solution
Ball ingestion fails	Rubber wheel wear / motor speed insufficient / distance threshold mis-calibrated	Inspect rubber wheels for visible wear and replace if needed; calibrate IR distance sensor thresholds; increase `pid_forward` output limits
Robot does not move	EP battery low / not in free motion mode	Check chassis battery indicator; verify `set_robot_mode("free")` has been called in the code
Cannot connect to Wi-Fi	Hotspot not started / IP address conflict	Verify Jetson `hostapd` service is running; restart Jetson and the iOS app
YOLO does not detect balls	Insufficient lighting / model not loaded / confidence threshold too high	Improve lighting; verify the model path is correct; lower `conf_thres` to 0.15

8. Appendix A: Quick Reference

8.1. Arduino Pin Mapping

See Table 2 in Section 3 for the complete pin assignments.

8.2. UDP Control Commands (iOS → Jetson)

The E-STOP, RECALL, AUTO, MANUAL, and JOYSTICK commands are described in Section 4. The full UDP packet specification is available in the CourtSweeper Theoretical Manual.

8.3. Wi-Fi Mode Selection

See Table 3 in Section 5 for AP/STA mode descriptions and SDK parameters.

8.4. CourtSweeper Source Code Structure

```
src/
├── chassis/ # Dual-PID chassis controller + ingestion state machine
│   └── chassis_ctrl.py
├── roller/ # Arduino firmware + Jetson serial driver
│   ├── roller_setting.ino
│   └── roller_drive.py
├── vision/ # YOLO inference + training + dataset collection
│   ├── video_node.py
```

```
│   ├──── cv_config.py
│   ├──── training/
│   │   ├──── dataset_collect.py
│   │   └──── model_training.py
│   └──── mlmodel/
│   └──── pt2onnx.py
├──── lidar/ # RPLidar driver
│   └──── lidar_parse.py
├──── distance/ # IR distance sensor subscriber
│   └──── distance_sub.py
└──── env_config.py # Global logging + debug configuration
```

8.5. Contact

1. **Open-Source Repository:** https://github.com/RamessesN/CourtSweeper
2. **RoboMaster-SDK-Ultra (forked SDK):** https://github.com/RamessesN/Robomaster-SDK-Ultra
3. **Issue Reporting:** Please submit bug reports or feature requests via GitHub Issues